Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study an intended semantics-preserving discretization for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so inference on the generated artifact reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax. We formalize the operational semantics and state the measurable weak compiler simulation required for soundness of Boolean queries; completing that simulation remains an open proof obligation. Generated programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation studies case studies combining continuous sampling, higher-order functions, and unbounded recursion and benchmarks where direct comparison with existing systems is possible.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 102 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies an intended semantics-preserving source-to-source discretization that converts, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\texttt{uniform}(0,1) < 0.5$ discretizes to $\texttt{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* names the intended equality of the relevant source and target probabilities. Our current result states that equality conditional on a measurable weak compiler simulation, which remains open. This differs from approximate discretization, which intentionally changes the original semantics according to a precision parameter and may instead provide error bounds or convergence guarantees.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

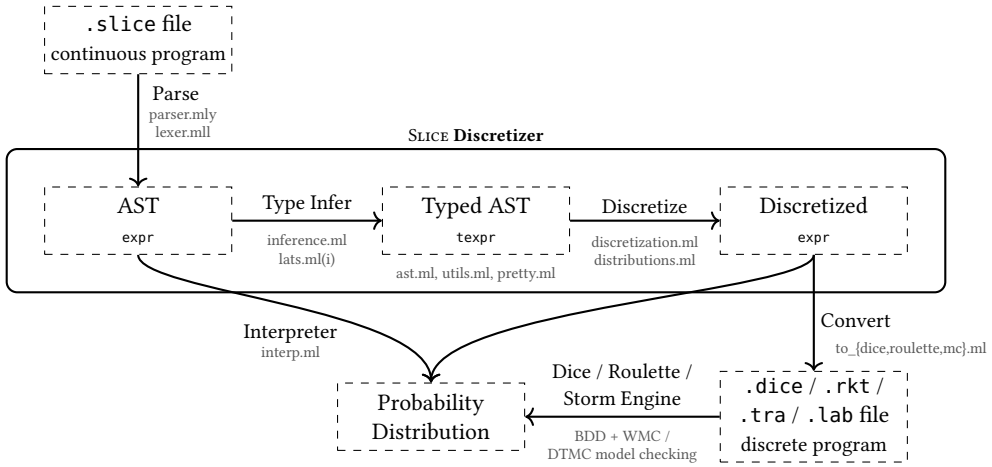


Fig. 1. Pipeline for discretization: typed Slice programs are transformed into discrete programs, then evaluated by discrete engines. Source-to-target equivalence is conditional on the weak compiler-simulation obligation in §6. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.
Type Inference (§4) Infer types automatically to guide discretization.
Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.
Soundness (§6) Prove semantic preservation of the discretization transformation with a coupling-style relational argument [7, 13, 48].
Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, there is no prior exact inference system that can cover programs *combining* all three of these features and (ii) case studies where Slice+Dice and Slice+Roulette show favorable scaling and empirical performance on new and existing benchmarks [22, 40].

2 Slice by Example

To build intuition for Slice, we start with simple examples demonstrating the key ideas, and then provide a more involved example containing higher-order recursion and lists in §2.6. Our pipeline automatically produces and analyzes discrete artifacts for all provided programs; source-to-target equality remains conditional on the weak compiler-simulation obligation in §6.

2.1 Branching on a Single Continuous Variable

Consider a program that samples a real value from a standard Gaussian distribution and then performs two nested comparisons: first checking whether the sample is less than 0.8, then comparing it to either 0.1 or 2.0:

```
let x = gaussian(0,1) in if x < 0.8 then x < 0.1 else x < 2.0
```

We discretize this program by replacing the real-valued sample with a discrete interval index, as shown below and in Figure 2:

```
let x = discrete(0.5398, 0.2483, 0.1891, 0.0228) in if x < 2 then x < 1 else x < 3
```

Here, **discrete**(0.5398, 0.2483, 0.1891, 0.0228) already represents all four observationally distinguishable outcomes of the Gaussian sample. The precise real value of x never matters to the rest of the program; only the interval in which it falls does. This is the regime we target: continuous quantities observed only through finitely many comparison outcomes. *Slice's goal is to detect such situations automatically.* We partition the continuous distribution into intervals that are indistinguishable to the rest of the program, then map every real to the interval it inhabits.

Our type system guides the transformation, in particular through a *cut set* — the set of comparison thresholds extracted from the program. A continuous distribution is annotated with its cut set, which records the values it is compared against. For our example, the type annotation is:

```
let x = (gaussian(0,1) : float[{<0.1,<0.8,<2}]) in if x < 0.8 then x < 0.1 else x < 2.0
```

Intuitively, the cuts $\{<0.1, <0.8, <2.0\}$ partition the real line $\mathbb{R}$ into four intervals $(-\infty, 0.1)$, $[0.1, 0.8)$, $[0.8, 2.0)$, $[2.0, +\infty)$, and only the interval containing the Gaussian sample x affects the rest of the computation. Slice uses these cuts to replace the continuous distribution

$$
\begin{array}{lr}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\quad \mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\quad \mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\quad \mid \textbf{discrete}(p_0, \ldots, p_n) & \text{discrete distribution} \\
\quad \mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 \mid e_1 ==_{\#n} e_2 & \text{comparisons} \\
\quad \mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\quad \mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\quad \mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\quad \mid \textbf{fun}\ x \to e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\quad \mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\quad \mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\quad \mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a discrete program (shown in §A.5); its equivalence to the source is conditional on the weak compiler-simulation obligation in §6. Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Probabilistic model checking then computes the result for the generated artifact: we use Storm [16], which reports 0.034474…. See §7.1 for details. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct $\textbf{observe}(e)$ conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{{<}x \mid x \in \mathbb{R}\} \cup \{{\le}x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{{\sim_1}x_1, \ldots, {\sim_n}x_n\}$ with $x_1 \le \ldots \le x_n$, we define the set $\text{Intervals}(B)$ of intervals it induces as expected, e.g., $\text{Intervals}(\{{<}0.1, {\le}0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{{<}c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{{\le}c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \le c$ places c on the left. Moreover, we let $\text{Intervals}(\emptyset) = \{(-\infty, \infty)\}$ and $\text{Intervals}(\top) = 2^{\mathbb{R}}$. We denote by $\text{Intervals}(B)_i$ the i-th interval of $\text{Intervals}(B)$, assuming $\text{Intervals}(B)$ is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.1 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$\tau ::= \textbf{int} \mid \textbf{bool} \mid \textbf{unit} \mid \textbf{fin}(n) \mid \textbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \textbf{list}(\tau)\ .$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\textbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\textbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type **float**[B; V] is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\textbf{fin}(n+1)$ where $n + 1 = |\text{Intervals}(B)|$ is the number of intervals induced by B. For instance, if $B = \{{<}0.5, {\le}0.8\}$, then the type is discretized to $\textbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains **float**.

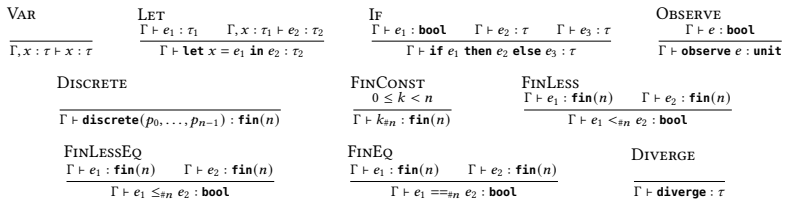


Fig. 7. Typing rules for general language constructs

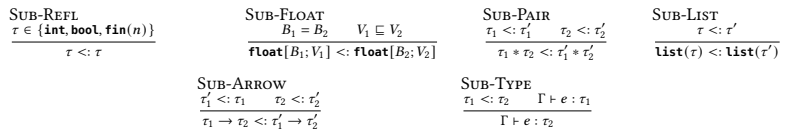


Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

$$C ::= \tau_1 <: \tau_2 \mid \mathsf{recovers}(B, V) \mid \mathsf{answers}_<(B, V_1, V_2) \mid \mathsf{answers}_\le(B, V_1, V_2) \mid \mathsf{has}(V, c)$$
$$\mid B_1 \sqsubseteq B_2 \mid B_1 = B_2 \mid V_1 \sqsubseteq V_2 \mid V_1 = V_2 \mid B_1 \equiv_\top B_2 \mid C_1, C_2$$

Fig. 9. Syntax for our constraint language. Constraints correspond to the semantic requirements in our type system: subtyping ensures type compatibility, $\mathsf{recovers}(B, V)$ ensures that distribution parameters can be recovered after discretization, $\mathsf{answers}_<(B, V_1, V_2)$ and $\mathsf{answers}_\le(B, V_1, V_2)$ ensure that comparison results can be determined after discretization, and $\mathsf{has}(V, c)$ ensures that constants are properly tracked.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\texttt{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The intended correctness argument separates into two obligations: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, the generated program satisfies the weak compiler simulation stated in §6. The second obligation is not discharged here.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, e *has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

The type inference rules closely mirror the typing rules in Figure 6, with two key differences. First, the inference rules generate a constraint set instead of having those constraints appear in the premises. Second, subtyping is handled in a syntax-directed way: rather than using a general subtyping rule, we bake the subtyping relation into each rule.

Example 4.1. To make the constraint generation process concrete, Figure 11 walks through §2.3. Each subexpression is annotated with type variables for the cut sets (B_i) and value sets (V_i), and the constraints produced at each step are shown explicitly.

4.2 Constraint Solving

Once constraint generation completes, we obtain a system of constraints over type variables, cut sets B, and value sets V. The constraint-solving phase proceeds in two steps:

(1) **Subtyping reduction**: We first process all subtyping constraints of the form $\tau_1 <: \tau_2$. Since subtyping for float types follows the rule $\texttt{float}[B_1; V_1] <: \texttt{float}[B_2; V_2]$ if and only if $B_1 = B_2$ and $V_1 \sqsubseteq V_2$, we reduce each subtyping constraint to corresponding equality and subset constraints on the cut and value sets. Subtyping elimination continues until only constraints among B's and V's remain.
(2) **Lattice constraint solving**: After subtyping reduction, we are left with constraints purely over cut sets and value sets: equalities, subset relations, and the specialized constraints $\mathsf{recovers}(B, V)$, $\mathsf{answers}_<(B, V_1, V_2)$, and $\mathsf{has}(V, c)$. These constraints are solved using a monotone fixed-point algorithm over the cut and value lattices.

- **Less-equal answerability** (answers$_{\leq}(B, V_1, V_2)$): Analogous to less-than. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. The monotonicity of the lattice operations ensures that this process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is the only way to satisfy all constraints. In other words, the discretization itself may be suboptimal in terms of the number of cuts.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\texttt{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\texttt{float}[B; V] \to \texttt{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression e : **bool** and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \texttt{bool}]\!]$: **bool** such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Theorem 6.1 states the resulting equality conditionally on the weak compiler-simulation obligation identified in the appendix; that obligation is not discharged by the current synchronous proof. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization aims to make that reduction explicit.

An impossible dispatch index or an invalid dynamic distribution tuple fails closed as (**observe** false); $0_{\#|B|+1}$. Observation failure discards that execution before the arbitrary finite fallback can be evaluated. This core-language term has zero mass in the interpreter, Dice, Roulette, and Storm. Explicit source-level runtime errors remain outside the generated target fragment and are rejected by backends that do not model them.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 isolates the weak-simulation condition needed to show that this non-local, type-mediated construction preserves semantics. The implementation represents the finite equality emitted by the final rule as the same $==_{\#n}$ expression shown here: its type checker requires two $\texttt{fin}(n)$ operands and its evaluator compares their finite indices. Thus the transformation, the formal target language, and the implementation use the same dispatch term. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \texttt{uniform}(e_1, e_2)$ of type $\texttt{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression e' : $\texttt{fin}(n+1)$ such that the probability that e evaluates to a value in Intervals$(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\texttt{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\text{Intervals}(B)_0 = (-\infty, 0.3) \qquad \text{Intervals}(B)_1 = [0.3, 0.5) \qquad \text{Intervals}(B)_2 = [0.5, \infty)$$

induced by B. Hence, all expressions of type `float[B; ·]` can be discretized to expressions of the finite type $\texttt{fin}(3)$, where $k_{\#3}$ represents the k-th interval Intervals$(B)_k$ induced by B for $k \in \{0, 1, 2\}$.

Now consider the discretized expression obtained from computing $\mathcal{D}[\![e : \texttt{bool}]\!]$, given by

```
let x = discrete(0.3, 0.2, 0.5) in (x <#3 1#3) || (x >=#3 2#3).
```

The probabilities appearing in **discrete**(...) are determined by the cumulative distribution function (CDF) of the uniform distribution over $[0, 1]$, ensuring that the probability of x evaluating to $k_{\#3}$ in $\mathcal{D}[\![e : \texttt{bool}]\!]$ *coincides* with the probability of x evaluating to a value in Intervals$(B)_k$ in e. Hence, the constants 0.3 and 0.5 in e can be replaced by the constants $1_{\#3}$ and $2_{\#3}$ in $\mathcal{D}[\![e : \texttt{bool}]\!]$.

Crucially, the *answers* constraint in the premise of the typing rules for comparisons (cf. Figure 6) ensures that B is a sufficiently fine-grained partition of $\mathbb{R}$, i.e., there is an injective function ToInts: $\{0.3, 0.5\} \to$ Intervals(B) that maps each constant $c \in \{0.3, 0.5\}$ to the unique interval in Intervals(B) containing c. This interval is determined by the cut convention: a cut $<c$ puts c in the interval to the right of the boundary, while a cut $\leq c$ puts c in the interval to the left.

Example 5.2. If the arguments to a uniform sampling are non-constant, the situation becomes more challenging. Consider the following typed expression e : **bool** (again only showing relevant types):

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \texttt{fin}(|B|+1) & \text{if } \tau = \texttt{float}[B;V] \text{ and } B \neq \top \\ \texttt{float}[B;V] & \text{if } \tau = \texttt{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

```
< (5 : float[{<5}; {5}])
```

Why can this expression be discretized even though the argument to the second uniform sampling instruction is non-constant? The typing information (x : `float[{<=10,<=20}; {10,20}]`) correctly tells us that x *can only take the values* 10 *and* 20. Consequently, x (and the **if**-expression generating it) can be discretized to a finite type **fin**(3) such that the probability of the resulting discrete expression evaluating to $0_{\#3}$ (resp. $1_{\#3}$) *coincides* with the probability of the continuous expression evaluating to 10 (resp. 20). Let us (partially) apply $\mathcal{D}[\![e : \texttt{bool}]\!]$, resulting in:

```
let x = if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3 in
( let v1' = D[[0 : float[{<=0}; {0}]]] in
    let v2'  = D[[x : float[{<=10,<=20}; {10,20}]]] in
    if v1' ==#2 D[[0 : float[{<=0}; {0}]]] &&
                v2' ==#3 D[[10 : float[{<=10,<=20}; {10,20}]]]
    then D[[uniform(0,10) : float[{<5}; T]]]
    else D[[uniform(0,20) : float[{<5}; T]]] ) <#2 D[[5 : float[{<5}; {5}]]]
```

The continuous **uniform**$(0, x)$-expression is replaced by a case distinction on the discretizations of **uniform**$(0, 10)$ and **uniform**$(0, 20)$, with each case executed with the correct probability 0.5. Since this reduces the remaining discretization process to discretizing expressions with *constant parameters only*, the remaining steps are completely analogous to Theorem 5.1.

Crucially, the *recoverability* constraints in the Continuous-rules from Figure 6 ensure that the parameter expressions of the respective sampling expressions can take only finitely many values, and that the cut sets are fine-grained enough to recover the values the parameter expressions evaluate to, as demonstrated in the preceding example.

A similar intuition applies to discretizable comparisons $e_1 < e_2$, where *both* expressions are non-constant but finite-cut. The *answers* constraints in the rule Less (resp. Less-Equal) ensure that at least one of e_1 and e_2 evaluates to only finitely many values, which enables discretization:

Example 5.3. Consider the following typed expression $(e_1 \leq e_2)$: **bool**, given by

```
(if discrete(0.5, 0.5) <#2 1#2 then
        uniform(0,10) : float[B; T] else 2.5 : float[B; {2.5}])
<=
(if discrete(0.5, 0.5) <#2 1#2
        then 2.5 : float[B; {2.5}] else 3.5 : float[B; {3.5}])
```

where $B = \{\leq 2.5, \leq 3.5\}$, and where e_1 : `float[B; T]` and e_2 : `float[B; {2.5,3.5}]`. Intuitively, e_1 samples from a mixture of a uniform and a dirac distribution, and e_2 samples from a finite distribution. Why can $e_1 \leq e_2$ be discretized even though neither e_1 nor e_2 is a constant? As suggested by its type, e_2 can take only finitely many values, namely $\{2.5, 3.5\}$. We apply $\mathcal{D}[\![\cdot]\!]$:

```
(if discrete(0.5, 0.5) <#2 1#2 then discrete(0.25, 0.1, 0.65) else 0#3)
<=
(if discrete(0.5, 0.5) <#2 1#2 then 0#3 else 1#3)
```

Notice that the probability of e_1 evaluating to some value in $(-\infty, 2.5]$ indeed *coincides* with the probability of the discretization of e_1 evaluating to $0_{\#3}$. For e_2, on the other hand, the probability of it evaluating to 2.5 (resp. 3.5) *coincides* with the probability of its discretization evaluating to $0_{\#3}$ (resp. $1_{\#3}$). For this reason, we obtain a semantically equivalent discretization of $e_1 \leq e_2$.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

The claim in this section concerns the *uniform fragment*: the syntax in Figure 5 without sequencing, non-strict comparisons, finite comparisons in source programs, or continuous samplers other than **uniform**. Write $Expr_\tau^{\mathsf{U}}$ for its well-typed expressions and $Expr_\tau^{\mathsf{U}+}$ for its operational closure under the finite comparisons and dispatch syntax emitted by Figure 13. The intended claim is restricted to closed e : **bool** in $Expr_{\texttt{bool}}^{\mathsf{U}}$:

*e : **bool** and $\mathcal{D}[\![e : \texttt{bool}]\!]$: **bool** (obtained from Figure 13) are semantically equivalent.*

The first step is to make formal sense of semantic equivalence through the monadic operational semantics in §6.1. Discretization decisions are driven by non-local, type-propagated information, so a local rule-by-rule argument is insufficient. The measurable weak compiler simulation needed to relate the two executions remains an open proof obligation.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics has to distinguish between non-termination and observation failure, which we realize as follows: The "missing" probability mass in $\langle e : \tau\rangle$ is the probability that e diverges. The probability of $\bot$ is the probability of encountering an observation failure. With this model, semantic equivalence indeed captures our sought-after correctness claim: If $\langle e : \texttt{bool}\rangle = \langle \mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \texttt{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\texttt{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; y : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \texttt{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \texttt{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \texttt{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type $\texttt{bool}$ (i.e., over $Expr_{\texttt{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type $\texttt{bool}$.

Equipped with this family of measurable spaces, we define the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). For instance, if e_1 is not a value, then[1]

$$[\![(\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\texttt{let}\ x=g\ \texttt{in}\ e_2:\tau_2):\tau_2} & \text{otherwise} \end{cases}}_{\text{standard probabilistic monadic bind over suitable Markov kernels}}.$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding $\texttt{let}$-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by $\texttt{observe}\ (\text{false} : \texttt{bool}) : \texttt{unit}$. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\texttt{observe}\ (\text{false} : \texttt{bool}) : \texttt{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding $\texttt{let}$-statement, i.e., also $[\![(\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. All details — including proofs that all monadic binds are well-behaved since they operate on suitable Markov kernels — are provided in §F.4.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= \lambda e'. \begin{cases} \delta_\bot & \text{if } e' = \bot \\ [\![e' : \tau]\!] & \text{otherwise} \end{cases}$$

This construction is well-defined since the functional $[\![\cdot]\!]$ itself gives rise to a suitable Markov kernel (Lemma F.39). We then define for every type τ the functional $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty} \langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing for every measurable set A over $Vals_\tau + \bot$ and upper-bounded by 1: executing a program for more and more steps can only increase the probability of terminating in a given set of values or an observation failure.

6.2 Soundness via Probabilistic Couplings

The generated target takes administrative steps that have no matching source step. The disabled text below records the earlier synchronous argument for comparison; the proposition after it states what currently follows.

Conjecture 6.1 (Weak-simulation closed Boolean semantic preservation). *Let $e : \texttt{bool} \in Expr^{\mathsf{U}}_{\texttt{bool}}$ be closed. A completed measurable weak-simulation theorem for the literal generated core term, together with administrative normalization and compatibility between weak-step iteration and the ordinary target terminal limit, would imply $\langle e : \texttt{bool}\rangle = \langle \mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}\rangle$.*

The required theorem has not yet been established. The earlier synchronous one-step argument is invalid for the emitted let/conditional dispatch because the target must take administrative steps while the source stutters. The conjecture records the desired consequence without treating the unspecified weak relation as already supplying finite-step couplings or a compatible terminal limit.

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

F.2 Measure Space on Expressions

To make the probabilistic semantics precise, we construct an appropriate measure space structure on the set of expressions by factoring out real numbers using the notion of what we call *expression skeletons*.

Definition F.7 (Expression Skeleton). An expression skeleton s is a syntactic expression obtained from a program by replacing each occurrence of a float constant with a hole of the form $\square$. The set of skeletons is generated by the grammar:

$$
\begin{aligned}
s ::= {} & \square \mid x \mid \text{true} \mid \text{false} \mid k_{\#n} \mid \textbf{diverge} \mid () \\
& \mid \textbf{let}\ x = s_1\ \textbf{in}\ s_2 \mid \textbf{if}\ s_1\ \textbf{then}\ s_2\ \textbf{else}\ s_3 \\
& \mid s_1 < s_2 \mid s_1 \leq s_2 \mid s_1 <_{\#n} s_2 \mid s_1 \leq_{\#n} s_2 \mid s_1 ==_{\#n} s_2 \\
& \mid \textbf{uniform}(s_1, s_2) \mid \textbf{gaussian}(s_1, s_2) \mid \textbf{exponential}(s) \mid \textbf{beta}(s_1, s_2) \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) \\
& \mid s_1\ \&\&\ s_2 \mid s_1\ ||\ s_2 \mid \text{not } s \mid \textbf{observe}(s) \mid s_1; s_2 \\
& \mid (s_1, s_2) \mid \textbf{fst}\ s \mid \textbf{snd}\ s \\
& \mid \textbf{fun}\ x \to s \mid s_1\ s_2 \mid \textbf{fix}\ f\ x := s \\
& \mid \texttt{nil} \mid s_1 :: s_2 \\
& \mid (\textbf{match}\ s\ \textbf{with} \mid \texttt{nil} \to s_{\text{nil}} \mid h :: t \to s_{\text{cons}}\ \textbf{end})
\end{aligned}
$$

We denote by $Skel$ the set of all expression skeletons, and by $holes(s)$ the number of distinct holes in skeleton s.

Example F.8. The expression $\textbf{uniform}(0.0, 1.0) < 0.5$ has the skeleton $\textbf{uniform}(\square, \square) < \square$ with three holes.

Definition F.9 (Filling a skeleton). Let $s \in Skel$ and let $\vec{r} = (r_1, \ldots, r_{holes(s)}) \in \mathbb{R}^{holes(s)}$. Define the expression $s[\vec{r}] \in Expr$ to be the result of replacing each hole $\square_i$ in s with the corresponding real constant r_i, for $1 \leq i \leq holes(s)$.

To recover a *typed* notion of skeletons, we extend the typing judgment to skeletons by adding a distinguished typing rule for holes.

Definition F.10 (Well-typed expression skeleton). Define a typing judgment for skeletons, written $\Gamma \vdash s : \tau$, with the understanding that all typing rules from expressions are preserved for skeletons, and that holes are typed as $\textbf{float}[B; V]$ according to the rule below:

$$\frac{}{\square : \textbf{float}[B; V]}$$

We denote by $Skel_\tau$ the set of all well-typed expression skeletons, where:

$$Skel_\tau := \{s \in Skel \mid s : \tau\}.$$

Remark F.11. Both $Skel_\tau$ and $Expr_\tau$ are to be understood as the set of all well-typed skeletons and expressions, respectively, *equipped with their full subtype information.*

For example,

$$\square : \textbf{float}[B; \{0.5\}] < \square : \textbf{float}[B; \{0.5\}] \quad \text{and} \quad \square : \textbf{float}[B; \{0.2\}] < \square : \textbf{float}[B; \{0.8\}]$$

are distinct elements of $Skel_{\textbf{bool}}$, while

$$0.5 : \textbf{float}[B; \{0.5\}] < 0.5 : \textbf{float}[B; \{0.5\}] \quad \text{and} \quad 0.2 : \textbf{float}[B; \{0.2\}] < 0.8 : \textbf{float}[B; \{0.8\}]$$

are distinct elements of $Expr_{\textbf{bool}}$.

In particular, both $Skel_\tau$ and $Expr_\tau$ are in general uncountable, since float subtype annotations of the form $\textbf{float}[B; V]$ may vary over uncountably many real values.

Definition F.12 (Decomposing an expression). The decomposition function

$$\text{dec} \colon Expr_\tau \longrightarrow \bigcup_{s \in Skel_\tau} \left(\{s\} \times \mathbb{R}^{holes(s)} \right)$$

is defined as the inverse of filling a skeleton, i.e., for every expression $e : \tau$,

$$\text{dec}(e) = (s, \vec{r}) \quad \text{such that} \quad e = s[\vec{r}].$$

The explicit recursive definition of dec is shown in Figures 26 and 27.

Definition F.13 (Hole assignments of a skeleton). For a given $s : \tau \in Skel_\tau$, define the set of hole assignments of s as:

$$A_{s:\tau} := \bigcup_{e:\tau \in Expr_\tau} \{\text{dec}(e : \tau)_2 \mid \text{dec}(e : \tau)_1 = s : \tau\} \subseteq \mathbb{R}^{holes(s)}$$

Intuitively, $A_{s;\tau}$ is the set of all real tuples that can be substituted into the holes of s to yield a well-typed expression of type τ.

Example F.14. Consider the skeleton

$$s : \tau = (\square : \textbf{float}[\{<3.5, <4.5\}; \top] \ < \ \square : \textbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \textbf{bool}.$$

This skeleton has two holes:

- The left hole has type $\textbf{float}[\{<3.5, <4.5\}; \top]$, so it may be instantiated by any real number.
- The right hole has type $\textbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]$, so it may only be instantiated by one of the values 3.5 or 4.5.

$\mathbf{e} : \boldsymbol{\tau}$	constraints	$[\![e : \tau]\!]$
$e_1 : \tau_1$ && $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{true}, e_2 = b_2 = \text{true}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \&\& g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \&\& e_2:\tau_2:\mathbf{bool}}$
$e_1 : \tau_1$ || $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{false}, e_2 = b_2 = \text{false}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{\text{true}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \| g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \| e_2:\tau_2:\mathbf{bool}}$
not $e : \tau : \mathbf{bool}$ $\tau = \mathbf{bool}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{\text{true}:\mathbf{bool}}$ $[\![e : \tau]\!] \gg\!= \lambda g.\delta_{\text{not } g:\mathbf{bool}}$
$(e_1 : \tau_1, e_2 : \tau_2) : \tau_1 * \tau_2$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(v_1,v_2):\tau_1 * \tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{(v_1:\tau_1, f):\tau_1 * \tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(f, e_2:\tau_2):\tau_1 * \tau_2}$
$\mathbf{fst}\ (e : \tau_1 * \tau_2) : \tau_1$	$e = (v_1, v_2)$ o.w.	$\delta_{v_1:\tau_1}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{fst}\ e:\tau_1}$
$\mathbf{snd}\ (e : \tau_1 * \tau_2) : \tau_2$	$e = (v_1, v_2)$ o.w.	$\delta_{v_2:\tau_2}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{snd}\ e:\tau_2}$
$(e_1 : \tau_1 \to \tau_2\ e_2 : \tau_1) : \tau_2$	$e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ g\ x := e, e_2 = v_2$ $e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ g\ x := e, e_2 \neq v_2$ o.w.	$\delta_{e[v_2/x]:\tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{e_1:\tau_1 \to \tau_2\ g:\tau_2}$ $[\![e_1 : \tau_1 \to \tau_2]\!] \gg\!= \lambda g.\delta_{f\ e_2:\tau_1:\tau_2}$
$\text{nil} : \mathbf{list}(\tau)$		$\delta_{\text{nil}:\mathbf{list}(\tau)}$
$(e_1 : \tau_1 :: e_2 : \mathbf{list}(\tau)) : \mathbf{list}(\tau)$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{v_1 :: v_2:\mathbf{list}(\tau)}$ $[\![e_2 : \mathbf{list}(\tau)]\!] \gg\!= \lambda g.\delta_{v_1:\tau_1 :: g:\mathbf{list}(\tau)}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{f :: e_2:\mathbf{list}(\tau):\mathbf{list}(\tau)}$
$(\mathbf{match}\ e : \mathbf{list}(\tau_1)\ \mathbf{with}$ $\quad \mid \text{nil} \to e_1 : \tau_2$ $\quad \mid h :: t \to e_2 : \tau_2$ $\quad \mathbf{end}) : \tau_2$	$e = \text{nil}$ $e = v_h :: v_t$ o.w.	$\delta_{e_1:\tau_2}$ $\delta_{e_2[v_h/h, v_t/t]:\tau_2}$ $[\![e : \mathbf{list}(\tau_1)]\!] \gg\!= \lambda g.\delta_{\mathbf{match}\ g\ \mathbf{with} \ldots:\tau_2}$

Table 2. Small-step operational semantics (continued). Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\perp$ in the sense of Definition F.29.

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. In particular, the last rule of Figure 13 denotes the displayed, capture-avoiding nest of core-language let-bindings, conditionals, Boolean conjunctions, and finite equalities. It is not a new primitive. Thus the transformation used in this appendix is exactly the source-to-source transformation in Figure 13; all its output forms occur in Figure 5 and are typed by the rules in Figure 7.

The usual syntactic equation $\mathcal{D}[\![e]\!][\,\mathcal{D}[\![v]\!]/x\,] = \mathcal{D}[\![e[v/x]]\!]$ does *not* hold for this transformation. Substitution can change which distribution clause applies: it can turn a dynamic parameter into a literal constant, or expose the invalid-parameter shortcut. What is needed is instead a semantic compiler-simulation lemma that permits the target to take finitely many deterministic administrative steps (let substitution, finite equality, Boolean conjunction, and conditional selection) while the source stutters. Establishing that lemma for every transformation clause remains an explicit proof obligation below.

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \perp)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\mathbf{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\perp)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

Then:

$$\begin{aligned} R_s &= \{\vec{r} \in A_{s;\tau_1} \mid \texttt{let } y = s[\vec{r}] \texttt{ in } e_2 \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid s'[(\vec{r}, \vec{c})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid s'[\phi_{\vec{c}}(\vec{r})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid \phi_{\vec{c}}(\vec{r}) \in A_{s'}\} \\ &= \phi_{\vec{c}}^{-1}(A_{s'}). \end{aligned}$$

Since $A_{s'} \in \mathcal{F}_{s';\tau_2}$ and $\phi_{\vec{c}}$ is measurable, we conclude

$$\phi_{\vec{c}}^{-1}(A_{s'}) \in \mathcal{F}_{s;\tau_1},$$

and therefore $B \cap Expr_{s;\tau_1} \in \mathcal{F}_{Expr_{s;\tau_1}}$ for all $s \in Skel_{\tau_1}$, giving $B \in \mathcal{F}_{Expr_{\tau_1}}$ as required.

Hence, $\{x \in Expr_{\tau_1} \mid g(x) > a\} \in \mathcal{F}_{Expr_{\tau_1}}$ for all $a \in \mathbb{R}$, so $g^{-1}((a,\infty) \cap [0,1]) \in \mathcal{F}_{Expr_{\tau_1}}$ for all $a \in \mathbb{R}$. Hence $g^{-1}(B) \in \mathcal{F}_{Expr_{\tau_1}}$ for every $B \in \mathcal{B}([0,1])$, and g is measurable.

Thus, $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel.

- All other cases analogous.

By Lemma F.30, the $\bot$-extension $f_\bot : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is also a Markov kernel, as required. □

Lemma F.39 (Small-step semantics is a Markov kernel). *For each type τ, the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a Markov kernel.*

Proof. By induction on the typing derivation of $e : \tau$. We show in each case that the map $e \mapsto [\![e : \tau]\!]$ is a Markov kernel.

- Consider $e = (\texttt{let } y = x : \tau_1 \texttt{ in } e_2 : \tau_2) : \tau_2$. The small-step semantics is:

$$[\![\texttt{let } x = e_1 \texttt{ in } e_2 : \tau_2]\!] = \begin{cases} \delta_{e_2[v/x]} & \text{if } e_1 = v \\ [\![e_1 : \tau_1]\!] \gg\!= \lambda g.\ \delta_{\texttt{let } x=g \texttt{ in } e_2} & \text{if } e_1 \neq v \end{cases}$$

  - If $e_1 = v$, By Lemma F.31 and Lemma F.37, $e \mapsto \delta_{e_2[v/y]:\tau_2}$ is a Markov kernel. Hence, $[\![\texttt{let } x = e_1 \texttt{ in } e_2 : \tau_2]\!] : Expr_{\tau_2} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel.
  - By the induction hypothesis,
  $$[\![e_1 : \tau_1]\!] : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_1} + \bot)$$
  is a Markov kernel. By Lemma F.38,
  $$\lambda g.\ \delta_{\texttt{let } x=g \texttt{ in } e_2} : (Expr_{\tau_1} + \bot) \to \mathcal{D}(Expr_{\tau_2} + \bot)$$
  is a Markov kernel. Hence by Lemma F.33, their composition
  $$[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\ \delta_{\texttt{let } x=g \texttt{ in } e_2} : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$$
  is a Markov kernel.
  Thus, $[\![\texttt{let } x = e_1 \texttt{ in } e_2 : \tau_2]\!] : Expr_{\tau_2} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a Markov kernel.
- Consider $e = \texttt{diverge} : \tau$.
  The small-step semantics is:
  $$[\![\texttt{diverge} : \tau]\!] = \lambda A.\ 0.$$
  By Lemma F.32,
  $$\lambda A.\ 0$$
  is a Markov kernel.
  Thus, $[\![\texttt{diverge} : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel.
- All other cases are analogous.

□

F.5 Small-Step Coupling Semantics

F.6 Weak compiler-simulation obligation

Let $\longrightarrow_{\mathrm{a}}$ contain only deterministic administrative target reductions: let substitution, finite equality, Boolean conjunction, and conditional selection. Write $\Longrightarrow_{\mathrm{a}}$ for its reflexive transitive closure. A sound proof for the literal generated term must construct a measurable relation R_τ such that

$$(e, \mathcal{D}[\![e]\!]) \in R_\tau \quad \text{and} \quad (e,t) \in R_\tau,\ e \longrightarrow \mu \implies t \Longrightarrow_{\mathrm{a}} ; \longrightarrow ; \Longrightarrow_{\mathrm{a}} \nu$$

with a coupling of μ and ν supported on R_τ. The symmetric clause permits target administrative steps while the source stutters. Terminal related Boolean values must be equal, and both sides must assign the same mass to observation failure. A completed theorem must additionally define the measurable weak target kernel, prove that generated administrative reduction is normalizing, and prove that the weak target terminal limit equals the ordinary target terminal semantics. Equality of the Boolean big-step distributions is only the expected consequence of all these obligations together.

The synchronous graph $\{(e, \mathcal{D}[\![e]\!])\}$ and its one-step projection lemma do not satisfy this obligation: the generated distribution dispatch takes several administrative steps, and clause selection is not stable under syntactic substitution. The former synchronous proof and tables are retained in the source above only as disabled historical material. This appendix therefore does not claim that the weak simulation has been proved. It identifies the exact missing lemma rather than treating a proof-only $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ operation as if it were emitted by the compiler.